

THURSDAY, SEPTEMBER 24 · MORNING

1 How a language model works · 2 Chat, CLI, and sandbox · 3 Prompting tips

Kerry Back

Four days

	WHEN	TOPICS
1	Thu 8:00	How a model works · chat, CLI, sandbox · prompting
2	Thu 1:30	Instructions · connectors and plugins · harnesses
3	Fri 8:00	Working with data
4	Sat 8:00	Creating docs · creating apps
5	Sat 1:30	Retrieval augmented generation · open source models
6	Sun 8:00	Creating agents · managing security risks

Every session has hands-on blocks in your own container. Bring a laptop and stay on wifi for four days.

This morning

How the model works

Tokens, vectors, one prediction at a time, and what training added on top.

Chat, CLI, and sandbox

The same model reached from a chat window, a command line, and a machine you were given.

Prompting tips

Planning, requiring code for numbers, and checking what comes back.

1 • HOW A LANGUAGE MODEL WORKS

Enough of the mechanism to explain what it is good at and where it fails.

Tokenization

Text is cut into pieces from a fixed vocabulary before the model sees any of it.

"The cat sat on the mat" → ["The", " cat", " sat", " on", " the", " mat"]

"Unbelievable" → ["Un", "believ", "able"]

Common words stay whole; rare and long words break into pieces. A typical vocabulary runs to about 100,000 tokens.

Numbers break on the same rules, which have nothing to do with arithmetic. 13,963,378.65 reaches the model as a handful of unrelated pieces.

The whole task

Given the tokens so far, predict the token that comes next. That is the entire operation.

Everything a model appears to do — answering, summarizing, writing code, refusing — is that one prediction, repeated.

The input can currently run to about a million tokens, roughly two-thirds of the Harry Potter saga. That context window has grown a thousandfold since GPT-3 in 2020.

Vectors

The model does not work with tokens. It works with numbers.

Each token is a list

“king” becomes 4,096 numbers. “queen” becomes 4,096 numbers that are close to them.

Tokens used in similar contexts end up near each other.

The space is mostly empty

There are far more possible vectors than there are tokens, so most points in the space name nothing.

The prediction can land between tokens.

A neural network is a function

THE IDEA

$y = 4x$ is a function. The 4 is a parameter.

Change it to 3 and you have the same structure with different behavior.

THE SCALE

Same idea with hundreds of billions of parameters, arranged as a transformer — the architecture published by Google researchers in 2017.

Training

What is learned

The vector for every token, and every parameter in the network.

Both are adjusted together to reduce prediction error on the training text.

What it takes

Trillions of words. Thousands of GPUs for weeks or months. Tens to hundreds of millions of dollars for one run.

Nothing is looked up or stored as fact. The arrangement captures how words are used, which is why it recalls common things reliably and rare things unreliably.

The pipeline

- 1 Tokenize the input.
- 2 Look up the vector for each token.
- 3 Feed the vectors into the network.
- 4 The network produces one output vector.
- 5 Find which token vectors are closest to it.
- 6 Emit a token, append it to the input, and start again at step 1.

Temperature

The network does not output one token. It outputs a probability for every token in the vocabulary. Temperature controls how that distribution is sampled.

SETTING	BEHAVIOR
Near 0	The highest-probability token every time. Repeatable.
Near 1	Sampled from the distribution. Different each run.
Higher	Low-probability tokens get picked. Text degrades.

In a chatbot, including Claude Desktop, temperature is fixed near 1 and you cannot change it. Through the API you can.

From prediction to conversation

A model trained only to predict the next token continues your text. It does not answer a question.

- 1 Role tokens. The text is wrapped in markers: `◁user▷` your question `◁assistant▷`.
- 2 Instruction tuning. The model is further trained on examples where what follows `◁assistant▷` is a useful reply.
- 3 Reinforcement learning from human feedback. Raters score replies, and the model is trained toward what raters preferred.

The mechanism did not change. What changed is which continuation is the likely one.

Reasoning models

The same trick applied again: train the model to write out its working before it answers.

How it was trained

Reinforcement learning on problems whose answers can be checked — mathematics, code, puzzles — rewarding the chains of tokens that arrived at a correct answer.

Why it helps

The model holds no state between tokens. The intermediate tokens are the only place a partial result can live, so writing them out is the computation.

The thinking you see may be a summary of a much longer chain, and you are charged for the whole chain as output tokens.

MOVES

Problems that decompose: multi-step arithmetic, code that has to satisfy several constraints, planning a sequence of actions.

More tokens at answer time, rather than a larger model, is the dial. You pay for it in latency and in output tokens.

DOES NOT MOVE

Recall of a fact it never learned, questions of taste, and anything where the first token was already right.

What thinking longer buys

What that explains

Fluent and wrong

Plausible text is exactly what the objective rewards. Correct text is a side effect of plausible text usually being correct.

Confident either way

Nothing in the mechanism distinguishes a recalled fact from a well-formed guess.

Arithmetic

Predicting the digits of a sum is a different operation from adding. Code does the second.

2 · CHAT, CLI, AND SANDBOX

The model is the same behind all three. What changes is what it can reach.

Ask the same thing three ways

“Here are four spreadsheets of orders. Which countries generated the most revenue?”

DOOR	WHAT HAPPENS
Chat in a browser	You upload the files. It answers in the window.
Claude Code on your laptop	It opens the files where they sit, writes code, saves a chart next to them.
Claude Code in a container	The same, on a machine that is not yours, holding nothing but what you put there.

Same model, different harness

THE MODEL

Identical in all three. Same weights, same training, same tendencies.

It has no memory of you, no filesystem, and no way to run anything.

THE HARNESS

The program around the model: what tools it offers, what it puts in front of the model each turn, and what it does with the answer.

Everything that differs between the three doors is here.

Sign in

EXERCISE 1 · 20 MINUTES

- 1 Go to `lab803.kerryback.com` and sign in with the username and password you were mailed.
- 2 Open the tab labelled Claude. You are at a command line inside a container that belongs only to you.
- 3 Type `ls` and then `pwd`. Note where you are.
- 4 Ask it, in English: *what files are in this folder, and what can you reach on this machine?*
- 5 Read the answer against what you saw in step 3.

Three tabs: a Claude terminal, a plain terminal, and a file browser. Port 8000 is yours for anything you serve.

Which tools each door offers

	CHAT IN A BROWSER	CLAUDE CODE
Write and run code	yes, in a sandbox	yes, on the machine
Read your files	what you upload	anything it has permission to open
Search and fetch the web	yes	yes
Run shell commands	no	yes
Install software	no	yes
Use your credentials	no	whatever is on that machine

Permissions

Ask

Every action that changes something stops for approval. The default.

Accept edits

File edits go through; commands still stop.

Bypass

Nothing stops. Reasonable in a container that holds nothing you would miss.

The permission mode and the tool list are two different limits. One asks you each time; the other decides what is possible at all.

What this door reaches

EXERCISE 2 · 25 MINUTES

- 1 Ask it to count how many files are in the container and how large they are altogether.
- 2 Ask it to install a Python package it does not have, then use it.
- 3 Ask a question it can only answer by running something — the size of the largest file, the number of lines in the longest one.

Watch what appears between your question and the answer: the command it chose, the output it got back, and what it did with output it did not expect.

Write down the first command that failed and what it did next.

WHY A MACHINE OF ITS OWN

It runs as you

An agent on your laptop has your filesystem, your network, your VPN, your saved logins, and your keys.

That is what makes it useful for your own work, and it is the reason the same setup is hard to hand to anyone else. There is no boundary between the task you asked for and everything else you can reach.

What a container changes

YOUR LAPTOP

Everything you have access to.

A mistake reaches the real thing.

YOUR CONTAINER

One home directory, the software installed for this course, and outbound internet.

A mistake reaches what you copied in.

Same agent, same model, same permission modes. The difference is the size of what it can touch.

Your container

Isolation	Its own filesystem, processes, users, and network namespace
Between students	Your container cannot reach any other student's
Sign-in	Your terminal and files sit behind your own login
Installed	Python, git, Node, and a set of skills we will use later in the week
Yours	<code>~/</code> and port 8000

First data question

EXERCISE 3 · 30 MINUTES

- 1 In `~/data` you have several spreadsheets of orders, customers, and products.
- 2 Ask for the schema first: what tables, what columns, how many rows, what date range.
- 3 Then ask one question that needs more than one file to answer.
- 4 Have it save a chart as a PNG and open it in the file browser.

Four or five of you will show what you got at the end of the block. Have the chart open and be ready to say which files the number came from.

3 · PROMPTING TIPS

Plan before executing

Ask for the approach before any work happens, and read it.

Prompt

Before you change anything: tell me how you would do this, what files you would touch, and what could go wrong. Do not start yet.

A plan can be corrected in a sentence. An execution has to be undone in every file it touched.

Numbers need code

If a number appears in prose and no code produced it, it was predicted.

Prompt

Compute this in Python and show me the code and the output. Do not report a number you did not compute.

The same rule applies to counts, percentages, dates, and anything summed across rows.

Verify claims

Ask what would falsify it

“What would have to be true for this to be wrong, and how would I check?”

Make it show its source

The file, the row count, the line of code. A citation you can open beats a summary you cannot.

Generator-critic

- 1 Get the first answer.
- 2 Start a fresh conversation and give it the answer without the reasoning that produced it.
- 3 Ask it to find what is wrong with it.
- 4 Take the objections back to the first conversation.

The second conversation has no stake in the first answer, which is the point of separating them.

- Accept a number in prose that no code produced
- Let it decide the method when you know the method
- Ask a question so broad that any answer looks like an answer
- Approve a plan you did not read
- Assume a second run gives the same result

What not to do

Planning and a critic

EXERCISE 4 · 30 MINUTES

- 1 Take the question from Exercise 3 and ask for a plan first. Read it before approving.
- 2 Run it. Require code for every number.
- 3 Open a second terminal, paste the result with none of the working, and ask what is wrong with it.
- 4 Bring the objections back and see which survive.

Write down one objection the critic raised that you would not have thought of.

This afternoon

Everything this morning was you typing into a fresh conversation. This afternoon is how to stop retyping it, and how to give the agent tools that are not on this machine.